Test Engine Handoff: Real-User Evaluation of AgentOS

Oct 13, 2026 · @Hassan

Purpose

The test engine should judge AgentOS the way a real user does: from the outside, with no knowledge of the code, against what the client asked for. This handoff redirects the engine toward that goal and says what to keep, change and build.

AgentOS is the Returned ACH resolution platform. A returned debit arrives from the payment processor (Stripe sandbox or a mock processor), AgentOS gathers the evidence, applies the policy, drafts a recommendation, and a person decides in the Control Center. AgentOS never debits or contacts the customer itself; it hands those steps to the client's own processes.

The engine's verdict should answer one question: does the product do what the client asked, for each kind of returned payment and each person involved? Whether the API rejects malformed input is already guaranteed by the code and is not the engine's job.

Where the engine is today

The last run reports 74 of 74 tests passing, but it proves little about the product: most tests check status codes, several accept the wrong answer, and one changes a live setting. Review of commit 3f358f5 against AgentOS main.

Issue	Example	Effect
Valid-request tests accept errors	4 "VALID" tests passed on 404 and 1 on 409	The resource never existed, so nothing was checked
Error tests accept the wrong status	The stale-hash test expects 409 and passed on 404	Real regressions would still pass
Reason-code tests only replay	Fixed traces such as TRC-R01-20260901 got 200 (duplicate), never 201 (created)	No case is created after the first run
Mock connector tests make up results	`mock_tester.py` sets PASS without sending a request	Coverage that does not exist

Issue	Example	Effect
Real connector tests accept 404	`real_tester.py` calls `/api/v1/info`, which AgentOS does not have	Always passes
The engine changes live settings	`PUT /api/v1/connectors/payment` as admin returned 200	Can change the default payment connector
The web UI accepts any website	CORS `*` and any target URL on `/api/discover`	A web page can start a run against your machine
Outcomes are never checked	No check of routing, role, recommendation or notice	The core of the product is untested
Sample data is unknown to the mock systems	`acct_ach_sample_101`, `cust_ach_sample_202`	Each case stops at a tool failure
Dates are fixed	2026-09-15, 2026-09-20	AgentOS now runs on the real date; future dates go to "needs input"

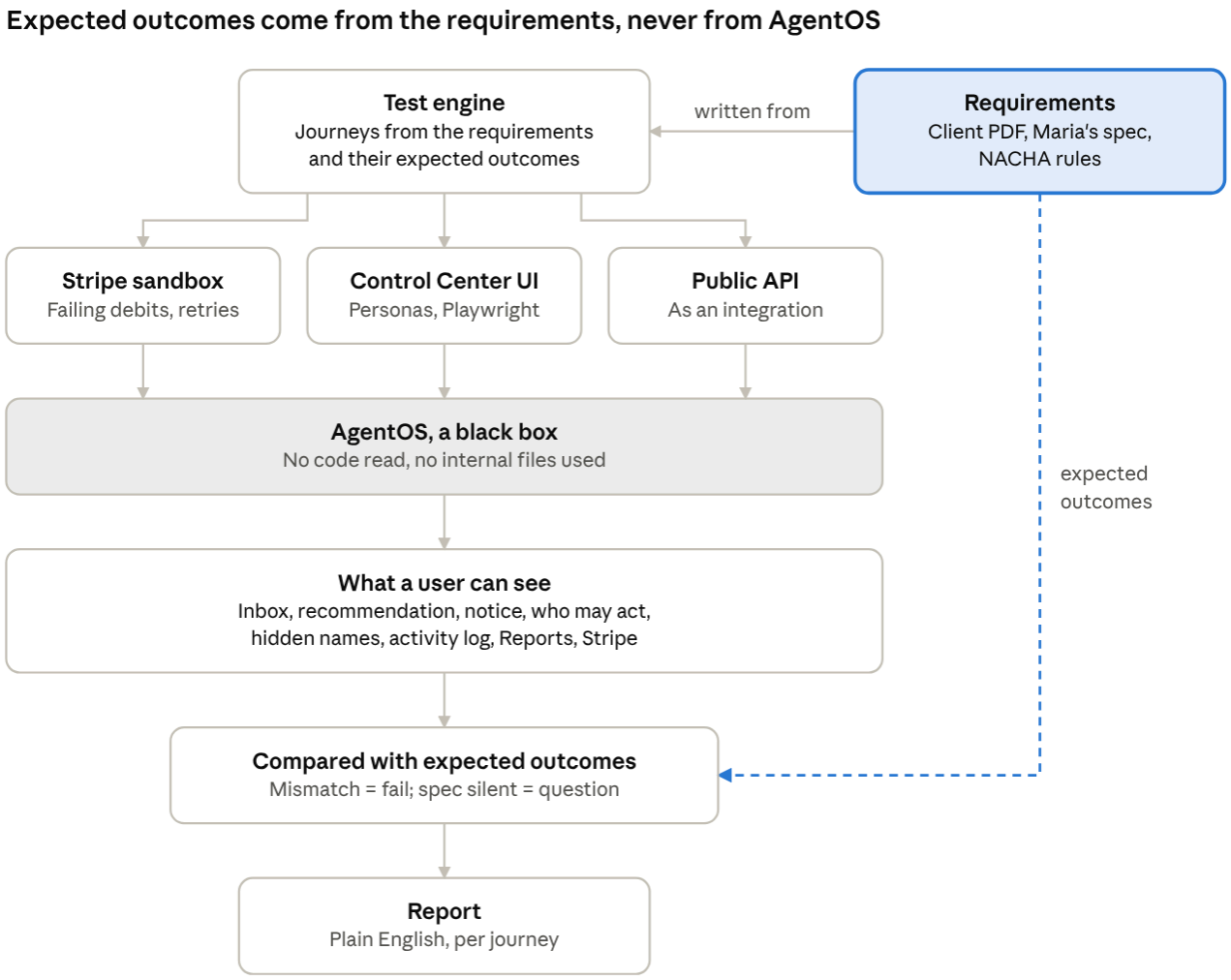
Worth keeping: the structure is clean, it never imports AgentOS code, it has its own tests, the reports are readable, the web UI binds to localhost, and the committed reports contain no customer data.

The approach

Treat AgentOS as a black box and test it the way its users meet it: real entry points in, visible outcomes out, judged against the requirements. Six principles:

1. **Expectations come from the requirements, never from AgentOS.** Write each expected result from the client PDF, Maria's spec and NACHA rules. Do not copy them from AgentOS code, its evaluation files or its OpenAPI field rules; an engine that copies the system can only confirm what was built, never catch a misread requirement.
2. **Enter the way real actors enter.** Returns come from the processor: create failing debits in the Stripe sandbox. People act in the Control Center UI, driven in a browser (Playwright) as each persona. Use the API only where a real integration would.
3. **Judge only what a user can see.** The inbox, the recommendation and its suggested date, the drafted notice, who may act, whether the customer's name is hidden, the activity log, the Reports numbers, and the Stripe dashboard.
4. **Test whole journeys, not single requests.** A journey runs from the returned payment to the case's final state, through every person who touches it.

5. **Report in plain English, journey by journey, with screenshots.** Where the requirements are silent and AgentOS made a choice, report a question, not a failure.
6. **Be safe by default.** A dedicated test environment, fresh test data on every run, no shared settings changed, and a cleanup at the end.



how the engine sees AgentOS · requirements in, visible outcomes out

The dashed line is the point: expected outcomes travel straight from the requirements to the comparison, never through AgentOS.

Journeys to cover

Start with journeys 1 to 3 (the client PDF's two scenarios and the retry outcome); the rest follow. Each expected outcome below is taken from the requirements, not from AgentOS.

#	Journey	How it starts	Who acts	Expected outcome
1	Scenario A: insufficient funds (R01), $89.00	A failing debit in the Stripe sandbox (insufficient funds)	Marcus, payments specialist	The case reaches Marcus. A retry is suggested 4 business days after the return, inside the retry window, with a drafted notice that names the date. Marcus can move the date. On approval the retry is handed to the client's payment process; AgentOS debits nothing.
2	Retry outcome	A retry of journey 1's debit in Stripe, once settling, once failing	none	Settled: the case records the retry as settled and Reports counts it. Failed again: the case reopens for a person.
3	Scenario B: no account (R03), $125.00	A no-account debit in the Stripe sandbox	Priya, reviewer	No retry is offered. A request for a new payment method is drafted, by email or SMS. On approval the case stays open, waiting for the customer, with a follow-up task; Priya closes it with a reason when the customer replies.

#	Journey	How it starts	Who acts	Expected outcome
4	Suspected fraud (R10) on a new account	A trigger for the fraud test account	Sam, fraud analyst	Urgent, and at the top of Sam's inbox. Retry is ruled out and no customer message is drafted. Blocking the account and stopping future debits are offered. The team is alerted, and Sam can open a fraud investigation.
5	Protected customer (bankruptcy)	A trigger for the protected test account	Dana, escalation owner	Nothing can be approved. Dana closes it without action, with a legal reason.
6	Duplicate event	The same Stripe event, or the same return, delivered twice	none	One case only; the second delivery is recognised as a duplicate.
7	Permissions and privacy	Any open case	A user with the wrong role; a role without identity access	The wrong role cannot decide. A role without identity access never sees the customer's name.
8	Reporting	After journeys 1 to 6	Admin	Return rates, retries that settled and dollars recovered reflect the runs.

Running it safely

Run only against a dedicated test instance of AgentOS, with fresh test data each run, and never change shared settings.

- **Environment.** A local stack (`make ui-api` plus `make ui`) or a staging deploy. Never the shared Neon database that the team's runtime uses.

- **Stripe.** Sandbox only, with test keys (`sk_test_`). Live keys must be refused. Use Stripe's failing test bank accounts to produce each return code.
- **People.** There is no sign-in yet. Act as a persona with the UI's identity switcher, or with the `X-User-Id` header on the API: `priya` (reviewer), `marcus` (payments specialist), `tom` (collections specialist), `dana` (escalation owner), `sam` (fraud analyst), `admin`.
- **Data.** A unique reference on every run, so each run creates new cases rather than replaying old ones. Compute every date from the day of the run; AgentOS runs on the real date.
- **Never.** Change connector settings, approve policies, or delete anything. Tests that must change data run only behind an explicit flag, such as `--allow-writes`, and only against a disposable database.
- **Predictable answers.** The local stack uses a recorded, deterministic model, so repeated runs give the same recommendations.
- **Cleanup.** Close the cases a run created, or throw the test database away.

Changes to the repository

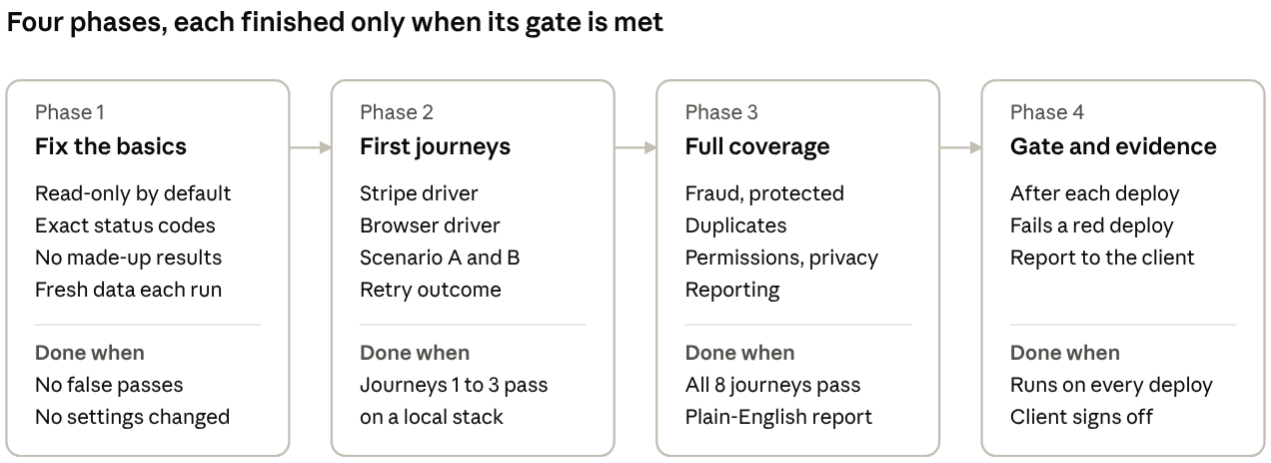
Keep the runner and the reports, replace the request-level tests with journeys, and add drivers for Stripe and the browser.

Action	What	Why
Keep	HTTP client and runner, CLI, report exporters (HTML, Markdown, JSON), localhost web UI	They work, and the reports are the engine's strongest part
Keep, smaller	OpenAPI discovery and a handful of input-validation tests	Useful for request mechanics; the code already guarantees most of it
Change	Validator: one exact status per test, plus checks on what the user sees	Today a wrong answer can pass
Change	Default to read-only; writes only behind `--allow-writes`	The engine changed a live connector setting
Change	Web UI: CORS limited to its own origin, a list of allowed target URLs	Any website could start a run today
Change	Test data: unique references per run, dates from today	Fixed data replays and goes stale
Add	Journey definitions written from the requirements, in a file people can read	One place for what "correct" means

Action	What	Why
Add	Stripe driver: create failing and retried debits in the sandbox	The real way a return arrives
Add	Browser driver (Playwright): act as each persona in the Control Center, take screenshots	The real way people meet the product
Drop	Made-up connector results in `mock_tester.py`	They report coverage that does not exist
Drop	LangGraph orchestration for a straight sequence	A plain sequence of steps is simpler
Drop	Committed `reports/` files and the empty `frontend/` folder	Generated files and an unused scaffold

Plan

Work in four phases and start the next only when the current gate is met; Phase 2 is the first result worth showing.



plan · 4 phases, each with its gate

No dates are set yet: agree them with the AgentOS team once Phase 1 is scoped. Phase 1 is mostly the fixes in the table above; Phase 2 adds the two drivers and the client PDF's scenarios.

Reference

Everything the engine needs from the outside, plus the questions still open.

What	Where
Control Center (UI)	`http://localhost:3000`
Runtime API	`http://localhost:8000/api/v1`; contract at `/openapi.json`; screens and endpoints listed in `docs/api/control-center.md`
Stripe webhook	`POST /api/v1/webhooks/returned-ach/stripe`, signed by Stripe; forwarded locally by `make stripe-listen`
Mock processor webhook	`POST /api/v1/webhooks/returned-ach/mock_processor`, header `X-Webhook-Signature: sha256=<HMAC of the body>`
Start the stack	`make ui-api` (runtime) and `make ui` (Control Center), from `core/products/agentos`
Make a Stripe return	`make stripe-return SCENARIO=a` (insufficient funds) or `SCENARIO=b` (no account)
Retry a Stripe return	`make stripe-retry TRACE=pi-… OUTCOME=settle` or `OUTCOME=bounce`
Test accounts in the mock systems	Trace numbers such as `TRC-SCENARIO-A`, `TRC-SCENARIO-B`, `TRC-R10-FRAUD`, `TRC-PROTECTED`, listed in `workflows/returned_ach/scenarios/` (test data, like Stripe's test bank accounts)
Requirements	The client PDF (Scenario A and Scenario B) and Maria's tool-by-tool spec

Open questions:

- [ ] R16 and R20 have no rule in the AgentOS policy and go to a person as unknown codes. Do the requirements say otherwise?
- [ ] Which environment does the engine run against after each deploy, and who owns it?
- [ ] Does Ijaz use the team's Stripe sandbox, or his own?
- [ ] Where are the reports published, and who reads them?